

“Collaborative Story/Screenplay Writing Platform with Version History”

CO4 ASSESSMENT TOOL 1

Submitted in the partial fulfilment for the Course of

CSA1016 - Software Engineering

to the award of the degree of

BACHELOR OF ENGINEERING

IN

Artificial Intelligence & Data Science

Submitted by

192424169- LITHISH KUMAR. R

Under the Supervision

of

Dr. K Anita Davamani



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

August 2025



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105



DECLARATION

I, **Lithish Kumar R 192424179**, of the **Department of Artificial Intelligence and Data Science**, Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Capstone Project Work entitled **“Collaborative Story/ Screenplay Writing Platform with Version History”** is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai
Date: 14/08/2026

Signature of the Students

Table of Contents

1. Introduction & Requirement Analysis	3
2. Pipeline Architecture & Workflow	4
3. Tools & Technology Selection	6
4. Automated Testing & Quality Integration	7
5. Deployment, Security & Rollback Strategy	9
6. Pipeline Execution Walkthrough & Documentation	10
7. Risk Assessment & Mitigation	12
8. Ethical & Compliance Considerations	12
9. Conclusion	13
Appendix A: Glossary of Key Terms	13
Appendix B: Roles & Responsibilities (RACI)	14

Document Scope

This design document is organized to mirror the evaluation rubric for the CI/CD Pipeline Design Exercise. Sections 1 and 2 analyze the assigned problem statement and design the pipeline architecture and workflow. Section 3 selects and justifies the tools and technologies used at every stage. Section 4 defines the automated testing and quality-integration strategy. Section 5 details the deployment environments, embedded security practices, and rollback mechanism. Section 6 walks through a concrete, end-to-end pipeline execution with supporting diagrams and screenshots. Sections 7 and 8 assess platform-specific risks and the ethical/compliance considerations relevant to a creative-writing collaboration product, and Section 9 concludes the document.

List of Figures

Figure 1. End-to-End CI/CD Pipeline Architecture⁴

Figure 2. Git Branching Strategy (GitFlow) and Pipeline Triggers⁵

Figure 3. Sample GitHub Actions Workflow Configuration (YAML)⁹

Figure 4. Multi-Environment Promotion Path with Blue-Green Deployment⁹

Figure 5. Sample CI/CD Pipeline Dashboard Run¹¹

List of Tables

Table 1. Tools & Technology Selection with Justification⁶

Table 2. Pipeline Quality Gate Pass Criteria⁸

Table 3. Requirement-to-Pipeline-Feature Traceability¹¹

Table 4. Risk Assessment & Mitigation Matrix¹²

1. Introduction & Requirement Analysis

1.1 Project Overview

The assigned problem statement is a **Collaborative Story/Screenplay Writing Platform with Version History** — a browser-based application that allows multiple authors, co-writers, and editors to draft, edit, comment on, and revise stories or screenplays together in real time. The platform behaves like a hybrid of a word processor and a source-control system for creative writing: every save is tracked as a revision, contributors can branch off alternate plot lines or scene rewrites, compare differences between drafts, and merge approved changes back into the canonical script. Because the system is collaborative, real-time, and revision-heavy, its CI/CD pipeline must be designed around frequent code changes, strict data-integrity guarantees for version history, and high reliability for concurrent multi-user sessions.

1.2 Core System Features Driving CI/CD Requirements

- **Real-time co-authoring:** Real-time collaborative text editor supporting simultaneous multi-user editing using Operational Transformation / CRDT synchronization (similar to Google Docs).
- **Version history engine:** Every edit is captured as an immutable revision; users can view a visual diff between any two versions and roll back to an earlier draft.
- **Branching & merging of scripts:** Ability to create a parallel draft (e.g., an alternate ending) and later merge it into the main script, with conflict resolution for overlapping edits.
- **Review & commenting workflow:** Inline comments, suggestion mode, and role-based permissions (Owner, Editor, Reviewer, Viewer).
- **Formatting & export engine:** Export to industry-standard screenplay formats (Final Draft .fdx, PDF) and story formats (.docx, ePub).
- **Account & collaboration management:** User accounts, project sharing, notifications, and payment/subscription handling for premium tiers.

1.3 Identified CI/CD Needs

Based on the above feature set, the following CI/CD requirements were identified for the platform:

1. **Frequent, low-risk integration:** Continuous Integration on every commit and pull request so that changes to the real-time synchronization engine and version-history logic are validated immediately, since regressions here directly corrupt user manuscripts.
2. **High test coverage for data-integrity-critical modules:** The diff/merge algorithm and revision database schema must be covered by rigorous automated tests before any deployment, as data loss in version history is unacceptable to writers.
3. **Zero-downtime deployment:** Because many authors may be actively editing at deployment time, the pipeline must support zero-downtime releases (blue-green or rolling deployments) so live editing sessions are not interrupted.
4. **Security and privacy compliance:** As a multi-tenant SaaS product handling user-generated creative content and personal data, the pipeline must integrate static/dynamic security scanning and dependency vulnerability checks at every build.
5. **Scalability validation:** The system must scale to support many concurrent WebSocket connections; pipeline-integrated load testing verifies this before every production release.
6. **Fast, safe rollback:** Any failed deployment must be detected via health checks and automatically rolled back to protect ongoing collaborative sessions.

1.4 Stakeholders & Environments

The pipeline serves three primary stakeholder groups — the development team (frequent commits to feature branches), the QA/review team (validating builds on staging before sign-off), and end users/writers (who consume only thoroughly tested production releases). Three corresponding environments are used: Development (continuous auto-deploy for internal testing), Staging (production-like, used for QA sign-off and security scanning), and Production (blue-green, customer-facing, high availability).

1.5 Assumptions & Constraints

The following assumptions and constraints scope the pipeline design and were used to guide tool and workflow selection:

- The platform's front end is built in React/TypeScript and its backend services in Node.js, packaged as Docker containers and orchestrated on Kubernetes; this assumption drives the choice of npm/Jest/Docker-centric tooling throughout the pipeline.
- The team operates in short, frequent sprints with multiple daily commits, so the pipeline is optimized for fast feedback (parallelized test stages, incremental builds) rather than long, infrequent batch releases.
- The product is hosted on a public cloud provider (assumed AWS) with managed Kubernetes (EKS) and container registry (ECR); an equivalent pipeline could be adapted to Azure/GCP with minimal changes to the deployment stage only.
- Given the platform stores original, unpublished creative writing, confidentiality and data-loss prevention are treated as first-class, non-functional requirements throughout the design — not an afterthought bolted on at release time.
- Budget and infrastructure constraints assume a small-to-mid-size engineering team (5–15 developers), which favors managed/SaaS tooling (GitHub Actions, SonarCloud/SonarQube, Snyk) over self-hosted alternatives that would need dedicated DevOps headcount.

2. Pipeline Architecture & Workflow

2.1 High-Level Pipeline Design

The pipeline is triggered automatically on every code push and pull-request merge and executes seven sequential stages — Source Code Management, Build, Automated Testing, Code Quality & Security Analysis, Packaging & Versioning, Staging Deployment, and Production Deployment — with monitoring and notification hooks attached to every stage. Figure 1 illustrates the full workflow.

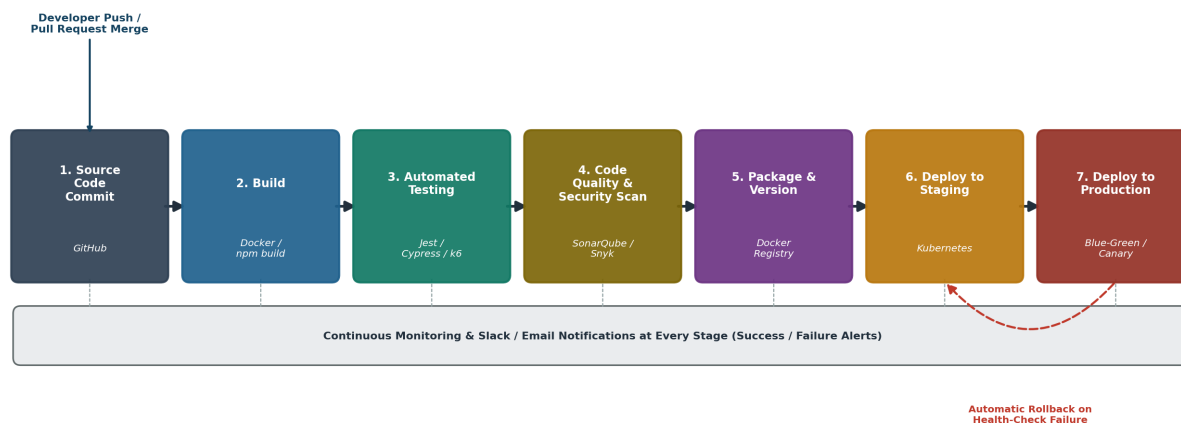


Figure 1: End-to-End CI/CD Pipeline Architecture for the Collaborative Story/Screenplay Writing Platform

2.2 Stage-by-Stage Workflow Explanation

- **Stage 1 — Source Code Management:** Developers commit to short-lived feature branches and open a Pull Request into develop. The PR automatically triggers the pipeline; direct pushes to main are blocked by branch-protection rules requiring at least one peer review and a passing pipeline run.
- **Stage 2 — Build:** The frontend (React-based editor) is built with npm, and the backend services (Node.js API, WebSocket synchronization service, diff/merge microservice) are compiled and containerized using multi-stage Docker builds to keep production images minimal.
- **Stage 3 — Automated Testing:** Unit, integration, and end-to-end test suites run in parallel containers, including dedicated tests that simulate two or more concurrent editors to validate the CRDT merge logic.

- **Stage 4 — Code Quality & Security Scan:** Static code analysis (linting, complexity, code smells), Static Application Security Testing (SAST), and open-source dependency vulnerability scanning run against the codebase; a failing quality gate stops the pipeline before packaging.
- **Stage 5 — Package & Version:** A Docker image is built, tagged with the Git commit SHA and semantic version (e.g., v1.2.0), signed, and pushed to a private container registry.
- **Stage 6 — Deploy to Staging:** The versioned image is deployed to the Kubernetes staging namespace automatically; smoke tests and Dynamic Application Security Testing (DAST) run against the live staging URL.
- **Stage 7 — Deploy to Production:** After a manual approval gate (QA/Release Manager sign-off), the image is promoted to production using a blue-green strategy; traffic is switched only after post-deployment health checks succeed, with automatic rollback on failure.

2.3 Branching Strategy Integrated with the Pipeline

The team follows a GitFlow-inspired branching model that maps directly onto the pipeline's trigger points, so that pipeline behaviour (which tests run, which environment is targeted) changes automatically with the branch type, as shown in Figure 2.

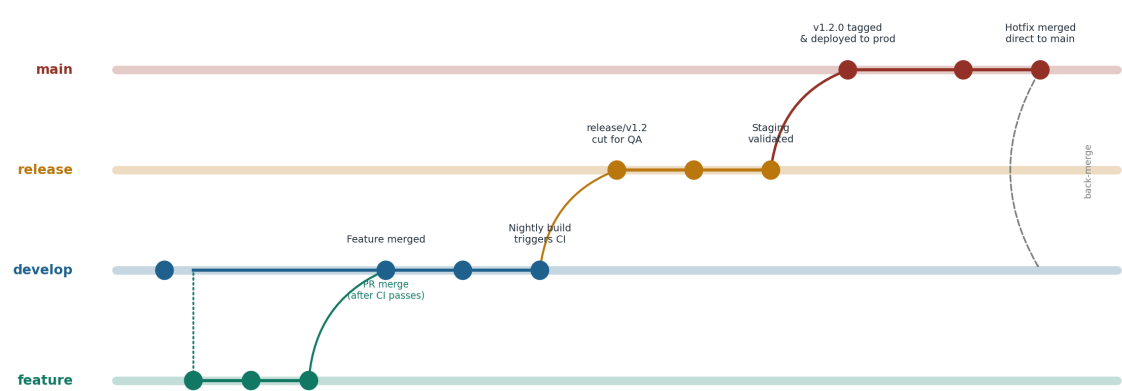


Figure 2: Git Branching Strategy (GitFlow) and its mapping to pipeline triggers

- **feature/*** — full unit + integration test run, deploys to a per-branch ephemeral preview environment for design/UX review.
- **develop** — nightly full pipeline run including E2E and load tests; auto-deployed to the Development environment.
- **release/*** — triggers the complete pipeline including security scans and deploys to Staging for QA sign-off.
- **main** — triggers production deployment through the blue-green strategy once release is approved and tagged.

2.4 Pipeline Triggers & Timing

Not every stage runs on every trigger — running the full pipeline (including load tests) on each of dozens of daily commits would slow the team down without adding proportional safety. Trigger scope is therefore tuned per event type, as summarized below.

Trigger Event	Stages Executed	Typical Duration
Push to feature/*	Build, unit tests, integration tests, preview deploy	~4–6 minutes
Pull request into develop	Build, unit + integration + E2E tests, code quality scan	~8–10 minutes
Nightly schedule on develop	Full suite including E2E, load tests, security scans, dev-environment deploy	~25–30 minutes
release/* branch cut	Full pipeline: all tests, SAST/DAST, staging deployment	~35–40 minutes

Trigger Event	Stages Executed	Typical Duration
Merge/tag on main	Package, production blue-green deployment, health-check bake, notification	~10–15 minutes

3. Tools & Technology Selection

The tools below were selected to match the platform's technology stack (React front end, Node.js real-time backend, containerized microservices on Kubernetes) and its specific risk areas — concurrent editing correctness, data integrity of version history, and continuous availability.

Pipeline Stage	Selected Tool(s)	Justification
Source Code Management	GitHub + GitHub Actions	Native, tightly-integrated CI triggers on push/PR events; built-in branch-protection rules, code owners, and PR review gating fit the GitFlow model without extra tooling overhead.
Build	Docker (multi-stage builds), npm / Node.js build toolchain	Multi-stage Docker builds keep runtime images small and reproducible across dev, staging, and production, eliminating "it works on my machine" issues for the real-time sync service.
Unit & Integration Testing	Jest (JS/TS), Supertest (API), Postman/Newman	Jest is the de-facto standard for React/Node projects with fast parallel execution and built-in coverage reporting; Supertest validates REST/WebSocket API contracts.
End-to-End Testing	Cypress / Playwright	Simulates real multi-tab, multi-user collaborative editing sessions in a real browser — essential for verifying the CRDT-based real-time sync and diff/merge UI.
Performance / Load Testing	k6	Scriptable, developer-friendly load testing that can simulate hundreds of concurrent WebSocket editing sessions to validate real-time scalability before release.
Code Quality Analysis	SonarQube	Provides continuous static analysis, code-smell detection, maintainability ratings, and enforces a quality gate that blocks the pipeline on regressions.
Security Scanning (SAST/SCA)	Snyk, OWASP Dependency-Check	Detects vulnerable open-source dependencies and insecure coding patterns early, before they reach a container image, supporting the CO4 emphasis on security.
Dynamic Security Testing (DAST)	OWASP ZAP	Runs automated penetration-style scans against the live staging environment to catch runtime vulnerabilities (e.g., XSS in comment fields) missed by static analysis.
Containerization & Registry	Docker, Amazon ECR	Ensures identical, versioned artifacts are promoted across environments; ECR integrates natively with the chosen Kubernetes deployment target and supports image scanning.
Deployment Orchestration	Kubernetes (EKS), Helm	Kubernetes provides self-healing, horizontal auto-scaling for WebSocket-heavy workloads and native support for blue-green/canary rollout strategies via Helm charts.
Infrastructure as Code	Terraform	Version-controlled, repeatable provisioning of staging/production infrastructure, reducing configuration drift between environments.
Secrets Management	HashiCorp Vault / AWS Secrets Manager	Centralizes database credentials and API keys outside of source control and pipeline logs, satisfying security best practice for a platform storing user content.

Pipeline Stage	Selected Tool(s)	Justification
Monitoring & Logging	Prometheus, Grafana, ELK Stack	Real-time dashboards and centralized logs are essential to catch synchronization errors or elevated error rates immediately after each deployment.
Notifications	Slack, Email (via GitHub Actions/Webhook)	Immediate pipeline status alerts keep the small development team informed of failures without needing to actively monitor the CI dashboard.

3.1 Alternatives Considered

For several stages, credible alternative tools were evaluated before settling on the choices in the table above; the reasoning behind each trade-off is summarized below so the selection is defensible rather than arbitrary.

- **CI orchestration — GitHub Actions vs. Jenkins:** Jenkins offers greater self-hosted flexibility but requires dedicated infrastructure and maintenance; for a small team, GitHub Actions' managed runners and native GitHub integration reduce operational overhead considerably.
- **CI orchestration — GitHub Actions vs. GitLab CI:** GitLab CI/CD was considered given its integrated container registry, but the team's existing codebase and review workflow already live on GitHub, making GitHub Actions the lower-friction choice.
- **Unit testing — Jest vs. Mocha:** Mocha/Chai is a viable alternative to Jest, but Jest's built-in snapshot testing, mocking, and coverage reporting reduce the number of additional libraries the team must configure and maintain.
- **E2E testing — Cypress vs. Selenium:** Selenium remains widely used, but Cypress's native browser execution and time-travel debugging significantly speed up authoring and diagnosing tests for the complex, real-time multi-tab scenarios this platform requires.
- **Orchestration — Kubernetes vs. Docker Swarm:** Docker Swarm was considered for its simplicity, but Kubernetes' maturity, wide ecosystem support for blue-green/canary rollout patterns, and horizontal auto-scaling make it the stronger long-term fit as the platform's user base grows.

4. Automated Testing & Quality Integration

4.1 Testing Pyramid for the Platform

Testing is organized as a pyramid, weighted toward fast, cheap unit tests and narrowing toward slower, high-value end-to-end and load tests, all fully automated and wired into the pipeline so no release proceeds on a broken quality gate.

- **Unit Tests (Jest):** Validate the diff engine, merge-conflict resolver, CRDT operation transforms, and formatting/export logic in isolation. Target: greater than 85% statement coverage on version-history and merge modules specifically, since these carry the highest data-integrity risk.
- **Integration Tests (Supertest / Newman):** Validate REST/WebSocket API contracts between the editor client, the collaboration service, and the persistence layer, including authentication/authorization checks for role-based permissions.
- **End-to-End Tests (Cypress):** Automate real user journeys: two simulated users editing the same scene concurrently, one user branching a draft and another merging it, comment threads, and export to .fdx/PDF, run headlessly in CI on every release-branch build.
- **Load & Concurrency Tests (k6):** k6 scripts simulate 200+ concurrent WebSocket editing sessions against staging to confirm latency and synchronization correctness stay within SLA before promotion to production.
- **Static Code Quality Gate:** SonarQube quality gate enforces thresholds on code coverage, duplication, cyclomatic complexity, and blocks merge on any new critical/blocker issue.
- **Security Testing (SAST/SCA):** Snyk and OWASP Dependency-Check scan every build for known CVEs in third-party packages (a real risk given the many npm dependencies of a rich-text collaborative editor); builds fail on high/critical findings.
- **Dynamic Security Testing (DAST):** OWASP ZAP runs an automated scan against the staging deployment to catch injection, XSS (particularly relevant given free-text comments and script content), and authentication flaws before production release.

4.2 Pipeline Quality Gates

A build is only allowed to progress to the next stage if all gates at the current stage pass. This fail-fast design means expensive stages (staging/production deployment) never run on code that has already failed a cheaper, earlier check. The table below summarizes the pass/fail criteria enforced at each gate.

Gate	Pass Criteria
Unit/Integration Tests	100% of unit and integration tests pass; coverage on version-history/merge modules remains above 85%.
End-to-End Tests	All critical user-journey scenarios (co-editing, branching, export) pass with no flaky retries exceeding threshold.
Code Quality (SonarQube)	Quality gate status is "Passed": zero new blocker/critical issues, duplication below 3%.
Security Scan (Snyk/OWASP)	No unresolved high or critical severity vulnerabilities in dependencies or container base image.
DAST (OWASP ZAP)	No high-risk alerts on the staging deployment (e.g., reflected/stored XSS, broken auth).
Production Health Check	Error rate and P95 latency remain within SLA for the full post-deploy bake period, else automatic rollback triggers.

4.3 Test Environment & Test Data Management

Reliable automated testing depends as much on how test environments and data are managed as on the tests themselves. The pipeline follows these practices to keep test runs fast, isolated, and representative of production behaviour:

- **Ephemeral test infrastructure:** Each pipeline run spins up isolated, disposable test containers (database, WebSocket service, API) via Docker Compose so parallel CI jobs never share state or interfere with one another.
- **Synthetic test data:** Test suites run against anonymized, synthetically generated scripts and screenplays rather than real user manuscripts, eliminating any risk of exposing genuine creative content or personal data during CI runs.
- **Deterministic seeding:** Before each E2E run, the test database is seeded from a fixed, version-controlled fixture set (sample projects, users, and revision histories) so tests are deterministic and reproducible across runs.
- **Parallel test isolation:** Independent, parallel test workers each get their own database schema/namespace, cutting overall pipeline duration by running unit, integration, and E2E suites concurrently rather than sequentially.

Figure 3 below shows a representative excerpt of the pipeline's actual GitHub Actions configuration, tying together the build, test, quality-scan, packaging, and multi-environment deployment jobs described in this section and the next.

4.4 Continuous Feedback to Developers

Fast, visible feedback is what makes the quality gates above sustainable in daily practice rather than a source of friction. Two practices keep the loop tight:

- **Inline PR status checks:** Every pipeline run posts a status check directly on the pull request, with failing stages linked straight to their logs so a developer can diagnose a failure without leaving GitHub.
- **Annotated quality reports:** SonarQube and coverage reports are published as PR comments summarizing exactly which lines or files caused a quality-gate failure, turning an abstract "build failed" into an actionable, line-level pointer.

```

ci-cd-pipeline.yml - story-platform

github/workflows/ci-cd-pipeline.yml
1  name: Story Platform CI/CD Pipeline
2
3  on:
4    push:
5      branches: [ develop, release/*, main ]
6    pull_request:
7      branches: [ develop ]
8
9  jobs:
10   build-and-test:
11     runs-on: ubuntu-latest
12     steps:
13       - uses: actions/checkout@v4
14       - name: Install dependencies
15         run: npm ci
16       - name: Run unit tests (Jest)
17         run: npm run test:unit -- --coverage
18       - name: Run E2E tests (Cypress)
19         run: npx cypress run --record
20       - name: SonarQube code quality scan
21         uses: sonarsource/sonarqube-scan-action@v2
22       - name: Snyk dependency & container scan
23         run: snyk test --severity-threshold=high
24
25   build-and-push-image:
26     needs: build-and-test
27     steps:
28       - name: Build Docker image
29         run: docker build -t story-platform:${{ github.sha }} .
30       - name: Push to ECR registry
31         run: docker push $ECR_REGISTRY/story-platform:${{ github.sha }}
32
33   deploy-staging:
34     needs: build-and-push-image
35     environment: staging
36     steps:
37       - name: Deploy to Kubernetes (staging)
38         run: kubectl apply -f k8s/staging/ --record
39       - name: Run smoke tests & DAST scan
40
41   deploy-production:

```

Figure 3: Representative GitHub Actions workflow configuration (ci-cd-pipeline.yml) implementing the build, test, quality-scan, package, and deploy jobs described above

5. Deployment, Security & Rollback Strategy

5.1 Multi-Environment Deployment Strategy

Code is promoted progressively through three environments, with automated gates and one manual approval before production, and a blue-green mechanism in production for zero-downtime releases, as shown in Figure 4. This progressive-promotion model exists because each environment answers a different question: Development confirms the code runs at all, Testing/QA confirms it behaves correctly against realistic multi-user scenarios, Staging confirms it is safe and secure enough to expose to real infrastructure, and Production confirms it performs well under real user load — with each gate catching classes of defects that are cheaper to fix the earlier they are found.

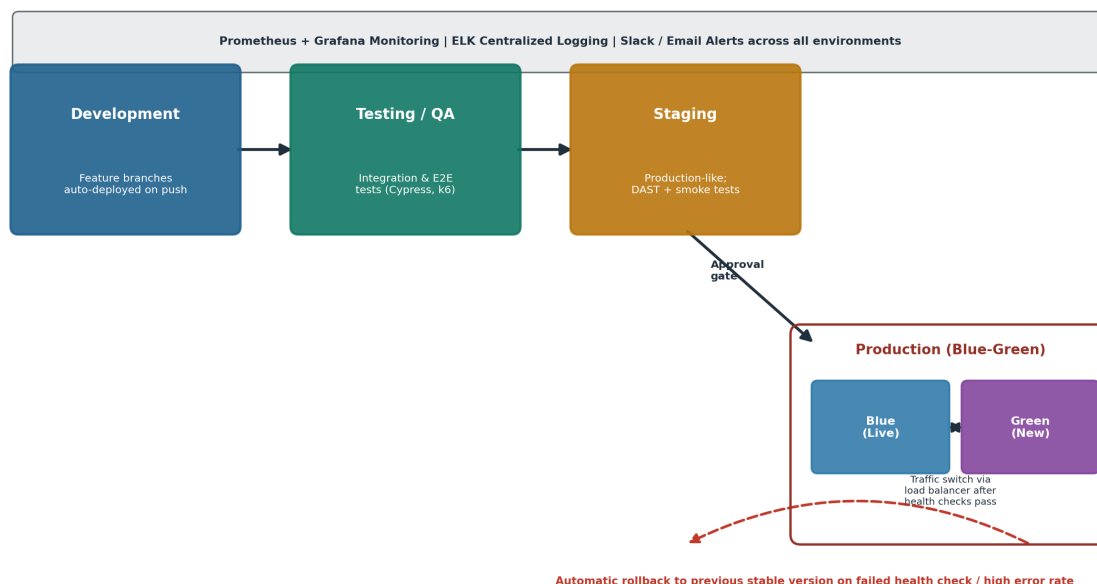


Figure 4: Multi-Environment Promotion Path with Blue-Green Production Deployment and Rollback

- **Development environment:** Every feature branch is auto-deployed to an ephemeral namespace for internal testing and design review; torn down automatically when the branch is merged or closed.
- **Testing/Staging environment:** A production-like environment used for full regression, load, and security (DAST) testing; requires QA sign-off before promotion.
- **Production (blue-green):** Runs two identical stacks — Blue (currently live) and Green (newly deployed). Traffic is switched at the load balancer only after the new version passes automated post-deployment health checks, giving true zero-downtime releases for authors mid-session.

5.2 Security Practices Embedded in the Pipeline

1. **Shift-left vulnerability scanning:** All source-to-image dependencies scanned by Snyk/OWASP Dependency-Check on every build; container images additionally scanned with Trivy before being pushed to the registry.
2. **Secrets management:** Database credentials, JWT signing keys, and third-party API keys are injected at deploy time from Vault/AWS Secrets Manager — never stored in the repository or pipeline YAML.
3. **Encryption in transit:** All traffic to the editor and API is served over TLS 1.3; pipeline includes a certificate-expiry check as part of the pre-deployment gate.
4. **Role-based deployment access:** Deployment credentials for staging and production are scoped separately, and production deploys require an explicit manual approval step from a release manager, satisfying least-privilege and separation-of-duties principles.
5. **Runtime security testing (DAST):** OWASP ZAP actively probes the staging build for injection and XSS vulnerabilities — a direct mitigation for the free-text, user-generated content (scripts, comments) this platform stores.

5.3 Monitoring, Alerting & Rollback

Prometheus scrapes application and infrastructure metrics (API latency, WebSocket connection count, error rate, merge-conflict rate) with Grafana dashboards for visualization, while the ELK stack centralizes logs from all microservices for fast root-cause diagnosis. A post-deployment automated health check monitors error rate and latency for a defined bake period (for example, 10 minutes) immediately after the blue-green traffic switch; if error thresholds are breached, the pipeline automatically reverts the load balancer to the previous stable (blue) stack without requiring human intervention, and a rollback notification is sent immediately.

All pipeline outcomes — success, failure, or rollback — trigger real-time Slack and email notifications to the development and release-management channels, ensuring the team can react within minutes even outside working hours.

5.4 Rollback Procedure — Step by Step

1. **Detect:** Prometheus alert rules continuously evaluate error rate, P95 latency, and WebSocket disconnect rate against the production "green" stack for the full post-deploy bake window.
2. **Alert:** If any threshold is breached, an automated Grafana/Alertmanager webhook fires, notifying the on-call engineer and simultaneously invoking the rollback pipeline job.
3. **Revert traffic:** The load balancer's target group is switched back to the previously live "blue" stack in seconds, since blue is left running (not torn down) until the bake period completes successfully.
4. **Preserve evidence:** The failed "green" deployment is left running in isolation (not destroyed) so engineers can pull logs and reproduce the failure without impacting live users.
5. **Notify:** A Slack message and email summarize what was rolled back, the triggering metric, and a link to the relevant Grafana dashboard and ELK log query.

6. Pipeline Execution Walkthrough & Documentation

6.1 End-to-End Walkthrough: Commit to Deployment

This section documents a single, concrete pipeline run to illustrate how the architecture in Section 2 behaves in practice, using an actual sample run captured from the CI/CD dashboard (Figure 5) for a release branch build.

1. **Trigger:** A developer merges a pull request implementing an improved diff-highlighting feature into release/v1.2; GitHub Actions detects the push and starts Run #482.

2. **Checkout & Build:** The commit (8f3a91c) is checked out in 4 seconds, dependencies installed via npm ci in ~41 seconds.
3. **Testing:** 214 Jest unit tests pass with 87.3% coverage; 42 Cypress end-to-end scenarios covering real-time co-editing, version diffing, and comment threads all pass.
4. **Quality & Security:** SonarQube reports a passing quality gate with zero blocker/critical issues; Snyk finds no high/critical dependency vulnerabilities.
5. **Packaging:** The Docker image story-platform:8f3a91c is built and pushed to the private ECR registry.
6. **Staging Deployment:** The image is deployed to the staging Kubernetes namespace; smoke tests confirm the service is healthy.
7. **Production Deployment:** After sign-off, the blue-green switch completes and production traffic is routed to the new "green" stack; a Slack notification confirms the successful release to the #deployments channel.

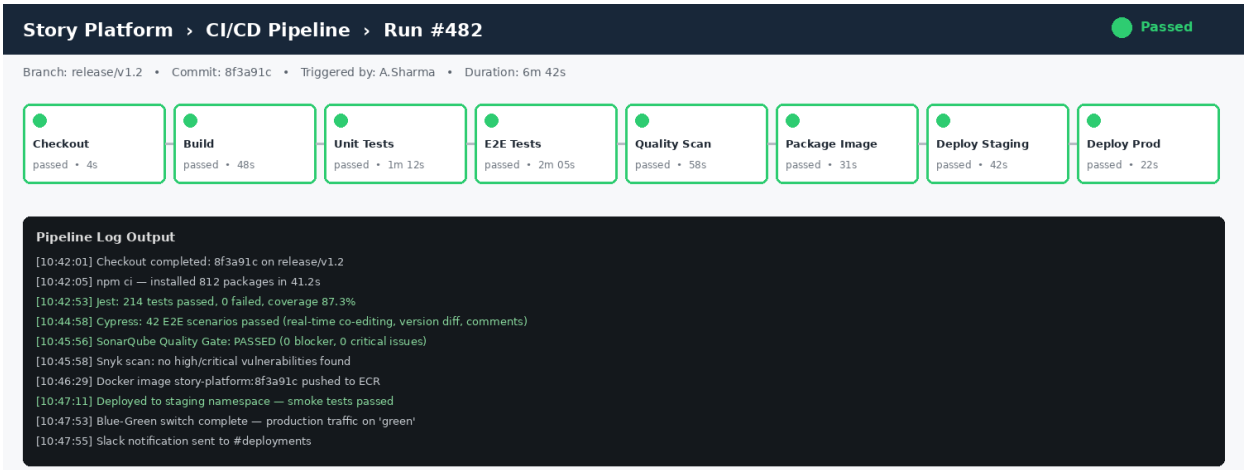


Figure 5: Sample CI/CD Pipeline Dashboard showing a completed run (all stages passed) with pipeline log output

6.2 Traceability Between Requirements and Pipeline Design

Identified Requirement	Pipeline Feature Addressing It
Data integrity of version history	High unit-test coverage requirement on diff/merge modules; SonarQube quality gate blocks regressions.
Concurrent multi-user editing correctness	Dedicated Cypress E2E scenarios simulate simultaneous editors; k6 load tests validate WebSocket concurrency.
Zero-downtime releases	Blue-green production deployment with automated health-check gating before traffic switch.
Security & privacy of user content	SAST (Snyk/OWASP Dependency-Check) and DAST (OWASP ZAP) integrated at build and staging stages; secrets never stored in code.
Fast recovery from bad releases	Automated rollback to the previous stable blue-green stack on failed health checks, with instant Slack/email alerts.

6.3 Key Pipeline Metrics Tracked

To continuously improve the pipeline itself, the team tracks a small set of DevOps performance indicators (aligned with common DORA metrics) on a rolling basis, reviewed at each sprint retrospective:

- **Pipeline duration:** Average of 6 minutes for feature-branch feedback and roughly 12 minutes for a full production deployment run, keeping developer feedback loops short.
- **Test flakiness rate:** Tracked separately per stage (unit, E2E, security scan) to flag flaky tests early; target below 2% flake rate for E2E suites.
- **Deployment frequency:** Multiple production deployments per week are supported safely due to the automated gates and blue-green rollback safety net.

- **Change failure rate:** Percentage of production deployments that trigger an automatic rollback; sustained target below 5%, investigated whenever exceeded.
- **Mean time to recovery (MTTR):** Time from a rollback-triggering alert to full traffic restoration on the previous stable version; target under 2 minutes given the automated blue-green switch.

7. Risk Assessment & Mitigation

The table below identifies the principal risks specific to a real-time, collaborative, version-history-driven platform and the corresponding pipeline-level mitigation designed into the CI/CD workflow described in the previous sections.

Risk	Impact	Pipeline Mitigation
Faulty merge/diff logic corrupts a writer's manuscript history	High — direct data loss, loss of user trust	Mandatory >85% coverage gate on merge/diff modules; dedicated concurrent-editing E2E scenarios must pass before any deploy.
Deployment interrupts an in-progress live editing session	Medium–High — user-visible disruption	Blue-green deployment with connection draining; traffic only switches after health checks pass.
Vulnerable dependency exposes user content (e.g., npm package CVE)	High — data breach, compliance exposure	Snyk/OWASP Dependency-Check gate blocks build on high/critical CVEs; scheduled nightly re-scan of production image.
XSS via user-generated comments/script text	Medium–High — account compromise	OWASP ZAP DAST scan on staging before every production release; output sanitization tested in E2E suite.
WebSocket service cannot scale under real collaborative load	Medium — degraded experience at peak usage	k6 load tests simulating 200+ concurrent sessions gate promotion from staging to production.
Bad release ships to all users simultaneously	High — widespread outage	Blue-green with automated health-check rollback limits blast radius and enables sub-minute recovery.
Secrets leaked via repository or CI logs	High — credential compromise	Vault/AWS Secrets Manager injects secrets at runtime only; pipeline configured to mask secret values in logs.

8. Ethical & Compliance Considerations

CO4 explicitly emphasizes ethics alongside security and reliability. For a platform that stores authors' original, often unpublished creative work, the CI/CD pipeline was designed with the following ethical and compliance commitments built into its automated checks rather than treated as manual afterthoughts:

- **Respect for user-generated intellectual property:** Test suites use synthetically generated scripts rather than real customer manuscripts (see Section 4.3), so engineers and automated tooling never gain incidental access to a writer's unpublished, potentially sensitive creative work during development or CI runs.
- **Data minimization and access control:** Every stage of the pipeline, and every environment, enforces role-based access so that only authorized services and personnel can read production data; deployment to production additionally requires named human approval, creating an auditable accountability trail.
- **Auditability:** Full audit logs of every pipeline run (who triggered it, what changed, what was deployed) are retained in the CI system and centralized logging stack, supporting transparent post-incident review and satisfying common data-protection audit requirements (e.g., GDPR-style accountability principles).
- **Integrity of authorship record:** Because version history is core to the product's trust proposition, the pipeline treats any risk of silently losing or corrupting a user's revision history as a release-blocking defect, not a lower-priority bug — reflecting an ethical obligation to preserve creators' work exactly as they left it.

- **Preventing misuse of the platform:** Dependency scanning and DAST checks are run on every release specifically to prevent the platform from being used as a vector to compromise user accounts or leak private drafts, in line with a duty of care to end users who trust the platform with unpublished work.

9. Conclusion

The CI/CD pipeline designed for the Collaborative Story/Screenplay Writing Platform with Version History automates the full journey from source-code commit to production deployment across seven well-defined stages, using a GitFlow-based branching model to control what level of testing and which environment each change reaches. The design directly addresses the platform's defining risks — data integrity of version history, correctness of real-time concurrent editing, and the need for zero-downtime releases — through a layered testing pyramid, mandatory code-quality and security gates (SAST, DAST, dependency scanning), a blue-green production strategy with automated health-check-driven rollback, and continuous monitoring with instant team notifications. Together these practices ensure that new features reach writers quickly, without ever compromising the reliability of their in-progress creative work, fulfilling the CO4 objective of implementing robust testing, quality assurance, and DevOps practices with an emphasis on security, reliability, and ethics.

Looking ahead, the pipeline is designed to evolve alongside the platform: canary releases could be layered on top of the existing blue-green strategy for even finer-grained rollout control, chaos-engineering drills could be scheduled against staging to validate resilience of the real-time synchronization service under induced failure, and the current manual production-approval gate could be progressively automated as confidence in the automated quality gates grows — without weakening the security, reliability, and ethical safeguards documented in this design.

Appendix A: Glossary of Key Terms

Term	Definition
CI/CD	Continuous Integration / Continuous Deployment — practices that automatically build, test, and release code changes frequently and reliably.
CRDT	Conflict-free Replicated Data Type — a data structure that allows multiple users to edit shared content concurrently and merge changes automatically without conflicts.
Blue-Green Deployment	A release strategy running two identical production environments (Blue and Green); traffic is switched to the newly deployed version only after it passes health checks, enabling zero-downtime releases and instant rollback.
SAST	Static Application Security Testing — analyzing source code for security vulnerabilities without executing the program.
DAST	Dynamic Application Security Testing — probing a running application for vulnerabilities by simulating attacks (e.g., OWASP ZAP).
SCA	Software Composition Analysis — scanning third-party/open-source dependencies for known vulnerabilities (e.g., Snyk).
Quality Gate	A defined set of pass/fail conditions (test coverage, code smells, vulnerabilities) that a build must satisfy before proceeding to the next pipeline stage.
Canary Release	A deployment technique that gradually shifts a small percentage of production traffic to a new version before a full rollout, limiting the impact of defects.
Rollback	Automatically or manually reverting a deployment to the last known-good version in response to a failure.
Infrastructure as Code (IaC)	Managing and provisioning infrastructure (servers, networks) through version-controlled configuration files (e.g., Terraform) instead of manual processes.

Term	Definition
GitFlow	A branching model that uses dedicated long-lived branches (main, develop) and short-lived supporting branches (feature, release, hotfix) to structure a project's development history.
Smoke Test	A quick, shallow test run immediately after deployment to confirm the application starts and its core functions work before deeper testing proceeds.
Bake Period	A defined observation window after a deployment during which health metrics are monitored before the release is considered fully successful.
MTTR	Mean Time to Recovery — the average time taken to restore service after a failure is detected.
DORA Metrics	A set of four key software-delivery performance indicators (deployment frequency, lead time for changes, change failure rate, and time to restore service) defined by the DevOps Research and Assessment team.

Appendix B: Roles & Responsibilities (RACI)

The table below clarifies who is **Responsible, Accountable, Consulted, or Informed** at each major pipeline stage, ensuring clear ownership across the development, QA, and release-management functions.

Pipeline Activity	Developer	QA / Reviewer	Release Manager	DevOps/SRE
Write & commit code	R, A	C	I	I
Review pull request	C	R, A	I	I
Maintain pipeline (YAML/ scripts)	C	I	I	R, A
Sign off staging build	I	R, A	C	I
Approve production release	I	C	R, A	I
Monitor post-deploy health	I	I	C	R, A
Execute rollback	I	I	A	R
Manage secrets & credentials	I	I	C	R, A

R = Responsible (does the work), A = Accountable (owns the outcome), C = Consulted (provides input), I = Informed (kept up to date).

Document Control

Document Title	CI/CD Pipeline Design — Collaborative Story/Screenplay Writing Platform with Version History
Prepared For	CO4 Assessment Tool 2 — CI/CD Pipeline Design Exercise, CSA10 / CSA1063 Software Engineering
Version	1.0
Total Pages	14